

RotaBook

Realtime Booking and Staff Rota App - Project Specification

1. Project Objective

Build a simple but production-ready full-stack app for managing bookings, availability slots and staff rota in realtime. The project is designed as a portfolio project to demonstrate a modern Firebase-based stack and serverless backend development.

Target skills to learn and show in the CV

- Next.js, React and TypeScript for the frontend
- Firebase Authentication for login and user identity
- Firestore for realtime NoSQL data storage
- Firebase Cloud Functions with Node.js and TypeScript for backend logic
- Firebase Hosting for deployment
- Firestore Security Rules for role-based access control
- Production-style data modelling, validation and error handling

2. Product Concept

RotaBook is a booking and staff rota platform for local service businesses such as gyms, clinics, personal trainers, barber shops, coworking spaces or beauty salons.

Main users

Role	What they can do
Admin	Create slots, manage bookings, create staff shifts, update booking status
Staff	View own shifts and assigned bookings
User	View available slots, create bookings, cancel own bookings

3. Final Tech Stack

Layer	Technology	Purpose
Frontend	Next.js, React, TypeScript	UI, routes, dashboard, forms and realtime data display
Backend	Node.js, TypeScript, Firebase Cloud Functions	Serverless backend logic and business validation
Database	Firestore	Realtime NoSQL database
Auth	Firebase Authentication	Email/password login and identity
Hosting	Firebase Hosting	Deploy frontend and serve production app
Security	Firestore Security Rules	Role-based permissions for users, staff and admin

4. MVP Features

4.1 Authentication

- Email/password registration and login
- Logout

- Protected routes based on authentication state
- Create a user document in Firestore after registration
- Default role is user; admin can be assigned manually in Firestore for the MVP

```
users/{userId}
{
  id: string,
  name: string,
  email: string,
  role: "admin" | "staff" | "user",
  createdAt: timestamp,
  updatedAt: timestamp
}
```

4.2 Admin Dashboard

Route: /admin/dashboard

- Total bookings
- Today bookings
- Available slots
- Pending, confirmed and cancelled bookings
- Realtime booking updates

4.3 Availability Slots

Admin can create and manage availability slots.

Routes: /admin/slots and /admin/slots/new

```
availability_slots/{slotId}
{
  title: string,
  serviceName: string,
  date: "YYYY-MM-DD",
  startTime: "HH:mm",
  endTime: "HH:mm",
  capacity: number,
  bookedCount: number,
  status: "available" | "full" | "cancelled",
  createdBy: string,
  createdAt: timestamp,
  updatedAt: timestamp
}
```

4.4 User Booking

Users can view available slots and book a slot through a Cloud Function.

Routes: /book and /my-bookings

- List available slots in realtime
- Book a slot using createBooking Cloud Function
- Prevent duplicate bookings for the same user and slot
- Cancel own bookings using cancelBooking Cloud Function
- See booking status updates in realtime

```
bookings/{bookingId}
{
  userId: string,
  userName: string,
  slotId: string,
  staffId: string | null,
  serviceName: string,
  date: "YYYY-MM-DD",
  startTime: "HH:mm",
  endTime: "HH:mm",
  status: "pending" | "confirmed" | "cancelled" | "completed",
  createdAt: timestamp,
  updatedAt: timestamp
}
```

4.5 Staff Rota

Admin can create shifts. Staff can view their own shifts.

Routes: /admin/shifts, /staff/schedule and /staff/bookings

```
staff_shifts/{shiftId}
{
  staffId: string,
  staffName: string,
  date: "YYYY-MM-DD",
  startTime: "HH:mm",
  endTime: "HH:mm",
  role: string,
  createdAt: timestamp,
  updatedAt: timestamp
}
```

4.6 Realtime Updates

- Admin sees new bookings without refreshing the page
- Users see slot availability change in realtime
- Staff see shift and booking updates in realtime
- Use Firestore realtime listeners with onSnapshot

5. Non-MVP Features

Do not build these in the first version: payments, Stripe, email notifications, push notifications, Google Calendar sync, multi-tenant accounts, file upload, chat and advanced analytics.

6. Architecture

```
User Browser
-> Next.js / React / TypeScript frontend
-> Firebase Auth
-> Firestore realtime listeners
-> Firebase Cloud Functions callable endpoints
-> Firestore database
-> Firebase Hosting production deployment
```

7. Repository Structure

```
rotabook/
  frontend/
    app/
      page.tsx
      login/page.tsx
      register/page.tsx
      admin/dashboard/page.tsx
      admin/slots/page.tsx
      admin/slots/new/page.tsx
      admin/bookings/page.tsx
      admin/shifts/page.tsx
      book/page.tsx
      my-bookings/page.tsx
      staff/schedule/page.tsx
      staff/bookings/page.tsx
    components/
      layout/
      auth/
      bookings/
      slots/
      staff/
      ui/
    lib/
      firebase.ts
      auth.ts
```

```

    firestore.ts
    functions.ts
  types/
    user.ts
    booking.ts
    slot.ts
    shift.ts

  functions/
    src/
      index.ts
      bookings/createBooking.ts
      bookings/cancelBooking.ts
      bookings/updateBookingStatus.ts
      slots/createSlot.ts
      staff/createStaffShift.ts
      utils/auth.ts
      utils/firestore.ts
      utils/validators.ts

  firebase.json
  firestore.rules
  firestore.indexes.json
  README.md

```

8. Cloud Functions Specification

8.1 createBooking

Field	Value
Type	Callable function
Input	{ slotId: string }
Output	{ success: true, bookingId: string }
Errors	unauthenticated, slot not found, slot not available, already booked, internal error

- Check that user is authenticated
- Read slot from Firestore
- Check slot exists and status is available
- Check bookedCount is lower than capacity
- Check user has not already booked this slot
- Create booking document
- Increment bookedCount
- Set slot status to full if bookedCount reaches capacity

8.2 cancelBooking

Field	Value
Input	{ bookingId: string }
Output	{ success: true }

- Check user authentication
- Read booking
- Allow cancellation if booking belongs to the user or requester is admin
- Set booking status to cancelled
- Decrement slot bookedCount
- Set slot status back to available if needed

8.3 updateBookingStatus

Field	Value
-------	-------

Access	Admin only
Input	{ bookingId: string, status: "confirmed" "cancelled" "completed" }

- Check user authentication
- Check admin role
- Update booking status
- Update updatedAt timestamp

8.4 createSlot

Field	Value
Access	Admin only
Input	title, serviceName, date, startTime, endTime, capacity

- Validate admin role
- Validate fields
- Create availability slot
- Set bookedCount to 0
- Set status to available

8.5 createStaffShift

Field	Value
Access	Admin only
Input	staffId, staffName, date, startTime, endTime, role

9. Frontend Routes

Route	Purpose
/	Landing page
/login	Login page
/register	Register page
/book	User slot booking page
/my-bookings	User bookings history
/admin/dashboard	Admin overview
/admin/slots	Admin slot list
/admin/slots/new	Create slot
/admin/bookings	Manage all bookings
/admin/shifts	Manage staff shifts
/staff/schedule	Staff rota page
/staff/bookings	Assigned staff bookings

10. Security Rules Concept

- Users can read their own profile and own bookings
- Admins can read and manage all users, slots, bookings and shifts
- Staff can read their own shifts and assigned bookings
- Authenticated users can read available slots
- Critical writes such as booking creation should go through Cloud Functions
- Block direct frontend writes that bypass business validation

11. Data Flow

Register flow

- User submits registration form
- Firebase Auth creates account
- Create user document in users collection
- Default role is user
- Redirect to /book

Booking flow

- User opens /book
- Frontend listens to availability_slots in realtime
- User clicks Book
- Frontend calls createBooking Cloud Function
- Function validates capacity and duplicates
- Function creates booking and updates slot
- UI updates in realtime

12. Deployment

Firebase services to enable

- Firebase Authentication
- Firestore Database
- Cloud Functions
- Firebase Hosting

Useful commands

```
npm install -g firebase-tools
firebase login
firebase init
firebase deploy
firebase deploy --only functions
firebase deploy --only hosting
firebase deploy --only firestore:rules
```

13. Environment Variables

Frontend .env.local

```
NEXT_PUBLIC_FIREBASE_API_KEY=
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=
NEXT_PUBLIC_FIREBASE_PROJECT_ID=
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=
NEXT_PUBLIC_FIREBASE_APP_ID=
```

14. Development Roadmap

Phase	Goal	Output
Phase 1	Setup Next.js and Firebase Auth	Login, register, user profile creation
Phase 2	Basic Firestore CRUD	Admin creates slots; users see slots realtime
Phase 3	Cloud Functions	Booking creation and cancellation through backend logic
Phase 4	Admin Dashboard	Admin manages bookings and statuses

Phase 5	Staff Rota	Staff shifts and schedule page
Phase 6	Security Rules	Role-based access and protected writes
Phase 7	Final Deploy	Firebase Hosting deployment and portfolio-ready README

15. Cursor Prompt

I want to build a production-ready portfolio project called RotaBook.

It is a realtime booking and staff rota app for local service businesses.

Tech stack:

- Next.js
- React
- TypeScript
- Firebase Auth
- Firestore
- Firebase Cloud Functions with Node.js and TypeScript
- Firebase Hosting
- Firestore Security Rules

The app must support three roles:

- admin
- staff
- user

MVP features:

- email/password authentication
- create user profile in Firestore
- admin can create availability slots
- users can view available slots in realtime
- users can book a slot through a Firebase Cloud Function
- Cloud Function must validate capacity and prevent duplicate booking
- users can view and cancel their own bookings
- admin can view all bookings and update booking status
- admin can create staff shifts
- staff can view their own shifts
- Firestore Security Rules must protect data by role

Please help me build this step by step.

Start with the project setup, folder structure, Firebase configuration, auth pages and user profile creation.

Keep the code clean, typed and easy to understand.

Explain each step because I want to learn Firebase, Firestore, Cloud Functions and serverless backend logic.

16. CV Entry After Completion

Project title

RotaBook - Realtime Booking and Staff Rota App

Stack

Next.js / React / TypeScript / Firebase Auth / Firestore / Node.js / Cloud Functions / Firebase Hosting

CV bullets

- Built a realtime booking and staff rota app using Next.js, TypeScript, Firebase Auth and Firestore
- Developed serverless backend logic with Node.js Cloud Functions to validate bookings, manage capacity and update booking statuses
- Implemented role-based access for admin, staff and users using Firestore Security Rules
- Deployed the full application with Firebase Hosting and production Firebase services